DS	**Session**	Tree 2	**Question Information**	Level 1 Challenge 74

Problem Description

You are given a weighted graph with N vertices and M edges. Find the total weight of its **maximum** spanning tree.

Constraints

- 1 <= T <= 20
- 1 <= N <= 5000
- 1 <= M <= 100 000
- 1 <= c <= 10 000

Input

The first line contains one integer T denoting the number of test cases. Each test case starts with a line containing 2 space-separated integer: N and M. Each of the following M lines contain description of one edge: three different space-separated integers: a, b and c. a and b are different and from 1 to N each and denote numbers of vertices that are connected by this edge. c denotes weight of this edge.

Output

For each test case output one integer - the total weight of its **maximum** spanning tree.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
1
3 3
1 2 2
2 3 3
1 3 4
```

EXPECTED OUTPUT

```
7
```

Test Case 2

INPUT (STDIN)

```
1
3 3
1 2 3
2 3 4
1 3 5
```

EXPECTED OUTPUT

```
9
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
int printheap(int N)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
2
```

Test Case 2

TOKEN COUNT

```
715
```

Test Case 3

NLOC

```
110
```

Code

C max_spanning_tree.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int u, v, w;
} Edge;

int cmp_desc(const void* a, const void* b) {
    return ((const Edge*)b)->w - ((const Edge*)a)->w;
}

int parent[5005];

int find_rep(int i) {
    if (parent[i] == i) return i;
    return parent[i] = find_rep(parent[i]);
}

int printheap(int N) {
    return N;
}

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;

    while (t--) {
        int n, m;
        scanf("%d %d", &n, &m);
        Edge* edges = (Edge*)malloc(m * sizeof(Edge));
        for (int i = 0; i < m; i++) {
            scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
        }

        qsort(edges, m, sizeof(Edge), cmp_desc);
        for (int i = 1; i <= n; i++) parent[i] = i;

        long long max_weight = 0;
        int count = 0;
        for (int i = 0; i < m; i++) {
            int root_u = find_rep(edges[i].u);
            int root_v = find_rep(edges[i].v);
            if (root_u != root_v) {
                parent[root_u] = root_v;
                max_weight += edges[i].w;
                count++;
                if (count == n - 1) break;
            }
        }

        printf("%lld\n", max_weight);
        printheap(n);
        free(edges);
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc max_spanning_tree.c -o max_spanning_tree
PS C:\C Programming> .\max_spanning_tree
1
3 3
1 2 2
2 3 3
1 3 4
7
PS C:\C Programming>